在 SkyStore 成本放置之上引入相似数据差分编码与副本–纠删混合冗余：增量式仿真评估

Incremental Evaluation of Delta Encoding and Replica–EC Hybrid Redundancy atop SkyStore-Style Cost Placement

作者姓名
单位院系
城市，国家
email@example.edu

摘要—云对象存储的多区域放置需要在存储占用费与跨区域传输费之间持续权衡。**SkyStore** 通过 **write-local**、**replicate-on-read** 与自适应 **TTL** 直方图，以货币成本为目标优化副本保留时长，但冗余形态始终为整对象副本，且未利用相似对象之间的 **delta** 冗余。**Finesse** 等相似检测方案可在块级发现 **resemblance** 并配合 **delta** 压缩节省空间，却未与多区域成本放置、**TTL** 驱逐语义联合建模。

本文在 **SkyStore** 类仿真栈（**TevictRanges** 基线）之上，提出 `PolicyMilpECnewV4`：保留 **TevictRanges** 的 **TTL**/驱逐语义，在写路径引入（**i**）基于 **manifest** 的差分编码（δ）体积与传输计费；（**ii**）副本与 (2+1) **RS** 纠删码（**EC**）的 **MILP** 布局快照比较。在与 **SkyStore** 相同请求行与价目表的 **IBM SNIA** 轨迹上，我们完成 $\delta\times$EC 的 2×2 因子分解、差分覆盖率 $p\in[0.05, 0.5]$ 扫描及 **T15/T65/T78/T79** 跨 **trace** 评估。

结果表明：（**1**）δ 带来独立、可测的 **incremental** 收益（**T79** 上 p=0.1 约 $-$**1.1%**，p=0.5 约 $-$**4.1%**），主要节省传输分项；（**2**）默认 **MILP** 从不选择 **EC**（ec_rate=0），强制 **EC** 略增成本；（**3**）δ 与 **EC** 无正向交互。本文定位为 **SkyStore** 之上的增量扩展；实验沿用 **SkyStore** 三账，并补充成本归因与覆盖率敏感性分析。

Index Terms—云存储；**SkyStore**；相似数据；差分编码；纠删码；混合整数规划；多区域放置；增量评估

I. 引言

现代应用日益跨多个可用区、地域乃至多云部署对象存储 [1]。用户在远端保留数据以缩短读延迟或满足合规时，必须在**存储账单**与**跨区域 egress** 之间做持续权衡。SkyStore 将问题形式化为**成本管理**：写路径 local-write，读 miss 时 replicate-on-read，并通过访问间隔直方图估计 ExpectedCost(TTL)，在“为副本付存储费”与“读 miss 再付传输费”之间自适应选择驱逐时长 [1]。仿真表明，相对固定 TTL 与工业基线，SkyStore 可在 IBM 类轨迹上取得显著节省 [1], [2]。

然而，SkyStore 在**冗余形态**与**数据相似性**两个维度存在空白：（**i**）**体积维度**——远端缓存按整对象体积 V 计费；纠删码虽可将单片存储降为 V/k，SkyStore 未在 TTL 框架内比较 replica 与 EC 的价目表分支 [3], [4]；（**ii**）**相似性维度**——真实工作负载存在大量非完全相同但高度相似的对象 [5]；对相似成员仅存储 delta 可在传输与存储上减少冗余，而 TevictRanges 基线不读取 similarity manifest，无法受益。

Ye 等提出的 Fast Variable-Grained Resemblance Data Deduplication [6] 指出：相似去重若一律细粒度切分，会显著增加元数据并损害性能；应在高冗余概率

区域细粒度、低概率区域粗粒度处理。受该思想启发，本文**不重新设计 CDC/特征提取**，而是将 Finesse 离线 manifest 作为可控旋钮（覆盖率 p），在已确定的多区域成本放置之上评估：当相似成员比例从低到高变化时，delta 与 EC 各自贡献多少——这是 incremental、可分解的实验路径。

A. 问题陈述

给定：SkyStore 类价目表（S_r 存储单价、N_{ij} egress 单价）、IBM 访问轨迹、Finesse 合并 manifest（差分率 r、成员 key 集合）。本文回答：（1）仅启用 δ、仅启用 EC、二者同时启用，相对 tevict 各节省多少？（2）当 p 从 0.05 增至 0.5，总成本如何变化？（3）在不同 SNIA trace 与访问模式 slice（typeA–D）上，结论是否稳健？

B. 主要贡献

- **系统：**在 TevictRanges TTL 语义不变的前提下，实现 V4 策略——δ 体积/传输计费 + 写路径 MILP 比较 replica 与 EC 分支（`PolicyMilpECnewV4` / `GurobiEc21MilpBuilder`）。
- **方法：**提出与 SkyStore 对齐的 δ×EC 2×2 ablation 及因子归因公式（内基线 `nodelta_rep`），并补充 $/req、transfer/aggregate GB、ec_rate 等 incremental 指标。
- **实验：**在 T79 上完成 $p \in \{0.05, 0.1, 0.2, 0.3, 0.5\}$ 扫描；在 T15/T65/T78/T79 上跨 trace 对比；报告 typeA–D 与 2×2 分解结果，并诚实报告 EC 未选中、T15 略负等边界情形。

C. 论文结构

第 II 节综述相关工作；第 III 节形式化模型；第 IV 节描述 V4 设计；第 V 节给出实验；第 VI 节总结。

II. 相关工作

A. 成本导向的多区域放置

SkyStore [1] 以货币成本为目标，用 TTL 直方图学习访问间隔，在多区域有向图上决策副本保留时长。SPANStore [7] 同样优化成本，但采用写时复制且不含 TTL 驱逐。Volley [8] 侧重延迟而非账单。TevictRanges（`PolicyTevictRangesNew`）复现 SkyStore 配对边直方图与 FP/FB 语义，是本文直接基线。本文不重新设计 TTL 学习，而在其输出的 T 快照上增加体积形态（V vs. V/k）与 δ 缩减体积 rV 的比较。

B. 相似数据去重

传统去重仅识别完全相同的块；Finesse [5] 通过分组秩特征加速 resemblance 检测，再对相似块做 delta 压缩。Ye 等 [6] 进一步指出元数据与块粒度的权衡，采用可变粒度减少不必要的细粒度检测。本文离线使用 Finesse manifest 标注 trace 中的 base/member 与差分率 r（均值约 0.007），在线仿真仅改变计费体积与传输量，与 SkyStore“同一 trace 行、同一定价” 的公平对比原则一致。

C. 纠删码与存储计价

(k, m) EC 将持久占用从 r 倍整对象降为约 $(k+m)/k$ 倍 [4]；Azure 等系统将 EC 作为默认冗余形态 [3]。Uluyol 等 [9] 从延迟–成本前沿论证 EC 相对纯复制的潜力。SkyStore 系策略优化保留多久；本文 MILP 在保留窗口已定时比较 replica 与 EC 的存储与放置 egress（不含编解码 CPU 费）。

III. 问题陈述

A. 计价模型

沿用 SkyStore 仿真栈：对象体积 V；区域 r 存储费率 S_r；边 (i, j) egress 费率 N_{ij}。7 天仿真窗口内，总成本 $C = C_{\text{storage}} + C_{\text{transfer}}$。整副本在区域 r 驻留 T 的存储费 $\approx V \cdot S_r \cdot T$；EC 数据片（$k$=2）单片 $\approx (V/k) \cdot S_r \cdot T$。$\delta$ member 有效体积 $\approx r \cdot V$，base 仍按 V；读 member 可能拉取 base+delta。

B. 决策边界

TevictRanges 决定 TTL T 与驱逐；MILP 不将驱逐时刻作为变量。MILP 仅在写路径直方图 warm 且跨 12 h 桶时触发，为每个 `issueRegion` 选择 replica 或 EC 落区。δ 由 trace 列与 manifest 决定；`allow_ec` / `force_ec_only` 为消融开关。

C. 因子分解（2×2）

表 I 给出四个实验角；归因公式为 $\Delta_\delta = C(\delta_rep) - C(nodelta_rep)$，$\Delta_{EC} = C(nodelta_ec) - C(nodelta_rep)$，交互 $= C(\delta_ec) - C(nodelta_rep) - \Delta_\delta - \Delta_{EC}$。

表 I
δ×EC 2×2 实验设计

corner	δ	EC
`nodelta_rep`	×	禁止
`nodelta_ec`	×	强制
`delta_rep`	✓	禁止
`delta_ec`	✓	允许

IV. 系统设计

A. 工作流程

参照 Ye 等 [6] 的上传/下载两阶段叙事，本文流程为：**（1）离线：**IBM trace + Finesse manifest → 合成 trace（`.delta.mc.aug`），控制 p = member key 占比；**（2）在线写：**write-local；直方图滚动 → 若触发 MILP，比较 replica/EC 分支成本 → 缓存布局决策；**（3）在线读：**miss 时按策略拉取（整副本 / base+delta / EC 恢复为整副本语义）；**（4）驱逐：**TTL 到期删除物理副本；与 tevict 语义对齐。

B. MILP 分支

Replica 分支：基地外至多一整副本，存储 $\propto V$，放置 egress $\propto V$。**EC 分支：**两条外传 data shard，存储 $\propto 2V/k$，放置 egress $\propto 2V/k$；读 miss 升级整对象恢复。当 T 较长且 V 较大时 EC 存储项理论上占优；实验显示在 IBM T79 + 3-reg FP 价目表下，传输与读恢复使 MILP 仍选 replica。

C. 实现要点

策略 `PolicyMilpECnewV4`（SimulatorV3, FP）；基线 `PolicyTevictRangesNew`（SimulatorV1）；求解器 `GurobiEc21MilpBuilder`；日志 `milp_solve_log_*.jsonl` 记录 ec_rate。

V. 实验评估

A. 数据集与设置

主轨迹为 SNIA IBM ObjectStoreTrace T79 typeA（114,227 请求，3 区域 FP，7 天窗口）。覆盖率扫描在同一 T79 上取 $p \in \{0.05, 0.1, 0.2, 0.3, 0.5\}$（464–4548 个 member key）。跨 trace 实验在 T15/T65/T78/T79 上重复（p=0.1）；T29（13M 请求）因 JVM 堆内存不足跳过。访问模式 slice typeA–D 共享同一 member 集，仅地理访问结构不同。

表 II
T79 typeA：差分覆盖率 p 扫描（USD，7 天窗口）

p	members	V4	tevict	vs. tevict	Δ transfer
0.05	464	273.08	275.19	−0.77%	−1.01
0.10	912	272.08	275.19	−1.13%	−1.93
0.20	1808	270.17	275.19	−1.83%	−3.63
0.30	2727	268.17	275.19	−2.55%	−5.44
0.50	4548	263.88	275.19	−4.11%	−9.27

表 III
T79 typeA p=0.1：2×2 分解（USD）

corner	total	vs. tevict	ec_rate
tevict	275.19	—	—
`delta_ec` / `delta_rep`	**272.08**	**−1.13%**	0%
`nodelta_rep`	274.22	−0.35%	0%
`nodelta_ec`	274.64	−0.20%	100%

Manifest 为 `finesse_merged_manifest.csv`，r 中位数 ≈ 0.005。V4 配置 `milp_ec_new_v4_3reg.yaml`；基线 `tevictranges_3reg.yaml`。tevict 不读 δ 列，保证与 SkyStore 公平对比。

B. 评价指标

沿用 SkyStore：total / storage / transfer cost（USD），requests，aggregate GB。**增量指标：**vs_tevict_%、vs_nodelta_rep_%、transfer/aggregate GB、cost/request、ec_rate、Δ_δ、Δ_{EC}。参照 Ye 等以 DCR 考虑元数据 [6]，本文以覆盖率 p 作为 resemblance 强度旋钮，避免与云账单口径混用。

C. 差分覆盖率扫描（*E13*）

表 II 给出 T79 上随 p 增加的节省；tevict 五档 total 恒为 275.19。V4 相对 tevict 从 −0.77%（p=0.05）增至 −4.11%（p=0.5）；全档 ec_rate=0。随 p 增加，节省近似单调上升；即使 p=0.5 仍约 4%，属于小幅 incremental 改进。

D. δ×EC 因子分解（*E12*）

表 III 给出 p=0.1 下的 2×2 结果。`delta_ec` 与 `delta_rep` 均为 272.08（−1.13% vs. tevict）；ec_rate=0。Δ_δ=−$2.14（transfer −1.93）；$\Delta_{EC}$=+$0.43；交互 ≈ 0。当前收益 $\approx$ 纯 δ；V4 相对 tevict 的额外 0.35% 来自 TTL/MILP 修复（`nodelta_rep`），与 δ 无关。

表 IV
跨 TRACE 对比（p=0.1，USD）

Trace	V4	tevict	vs. tevict
T65	6.56	6.59	−0.42%
T78	24.18	26.41	**−8.45%**
T79	272.08	275.19	−1.13%
T15	77.06	76.56	+0.65%

表 V
访问模式 SLICE（p=0.1，T9FIX，USD）

type	V4	tevict	vs. tevict	特点
A	272.08	275.19	−1.13%	写密集
B	208.21	209.74	−0.73%	区域感知
C	217.27	218.91	−0.75%	读集中
D	270.88	273.94	−1.12%	多读

E. 跨 trace 与访问模式

表 IV：T78 节省 8.45%（storage 13.56 vs. 15.68）；T15 略负（+0.65%），说明 δ 收益依赖 trace 结构。表 V：typeA–D 上 V4 相对 tevict 约 −0.7% 至 −1.1%；读集中的 typeC transfer 占比仅 ≈19%，绝对节省更小。

F. 讨论

本文定位为 SkyStore 之上的**增量**扩展：δ 带来可测但有限的成本下降；EC 在默认 MILP 下未被选中。参照 Ye 等 [6] 对粒度参数的扫描讨论，我们以 p 为旋钮展示 resemblance 强度与账单之间的单调关系。9-reg 与 force EC 实验表明：force EC 仅 +\$0.1 量级；高 r manifest（×10）在 p100 覆盖下仅 +\$0.16，说明**覆盖范围**比单点放大 r 更关键。

VI. 结论

本文在 SkyStore 类仿真栈上，对相似数据差分编码与副本–EC MILP 布局做增量式联合评估。核心发现：（1）δ 有效但幅度有限——T79 上 p 从 0.05 到 0.5，相对 tevict 从约 0.8% 增至 4.1%，主要节省传输；（2）EC 在当前配置下不被选用，单独 force EC 略增成本，与 δ 无协同；（3）实验与 SkyStore 同一 trace、同一三账，增量指标与 2×2 归因支持小步可测的论文叙事。

未来工作包括：更大 p 与更高 r manifest 联合扫描；6/9 区域价目表下是否出现 ec_rate>0；T29 分段仿真；读路径 EC 增量恢复以降低读放大。

致谢

（待填）

参考文献

[1] S. Liu, X. Mo, M. Hershcovitch, H. Zhang, A. Cheng, G. Girmonsky, G. Vernik, M. Factor, T. Bang, S. Ponnapalli, N. Crooks, J. E. Gonzalez, D. Harnik, and I. Stoica, "SkyStore: Cost-optimized object storage across regions and clouds," *Proceedings of the VLDB Endowment*, vol. 18, no. 1, pp. 1–14, 2025, arXiv:2502.20818.

[2] O. Eytan, D. Harnik, E. Ofer, R. Friedman, and R. Kat, "IBM object store traces (SNIA IOTTA trace set 36305)," in *SNIA IOTTA Trace Repository*, 2019.

[3] C. Huang, H. Simitci, Y. Xu, S. Smallberg, B. Calder, J. Li, and A. Patterson, "Erasure coding in windows azure storage," in *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, 2012, pp. 15–26.

[4] J. S. Plank, M. Blaum, and J. L. Hafner, "Erasure codes for storage systems: A primer," in *6th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2013.

[5] W. Tian, X. Ye, J. Tang, R. Li, and Z. Xu, "Finesse: Efficient similarity detection for cloud storage," in *IEEE International Conference on Communications (ICC)*, 2020, representative citation for resemblance detection.

[6] X. Ye, J. Tang, W. Tian, R. Li, W. Xiao, Y. Geng, and Z. Xu, "Fast variable-grained resemblance data deduplication for cloud storage," in *IEEE International Conference on Communications (ICC)*, 2021.

[7] Z. Wu, M. Butkiewicz, D. Perkins, E. Katz-Bassett, and H. V. Madhyastha, "SPANStore: Cost-effective geo-replicated storage spanning multiple cloud services," in *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, 2013, pp. 292–308.

[8] S. Agarwal, J. Dunagan, N. Jain, S. Saroiu, A. Wolman, and H. Bhogan, "Volley: Automated data placement for geo-distributed cloud services," in *Proceedings of the 7th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2010, p. 2.

[9] M. Uluyol, A. Huang, A. Goel, M. Chowdhury, and H. V. Madhyastha, "Near-optimal latency versus cost tradeoffs in geo-distributed storage," in *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 157–180.